- $E' \leq E$ to denote that the sequence E' is a prefix of the sequence E.

To simplify the presentation, we first, in Sections 3 to 7, make the assumption that a process does not disable another process, i.e., if p is enabled and another process q performs a step, then p is still enabled. We formalize this as the following assumption, so that it can be referenced in the subsequent exposition.

Assumption 3.1. *If $E \vdash p$ and $E \vdash q$ with $p \neq q$, then $E \vdash p.q$.*

Assumption 3.1 is valid for concurrent programs that communicate via message passing. I.e., it is valid for programs that follow the actor model of concurrency and Erlang programs in particular. However, note that even in such programs a process can disable itself, e.g., after a step such that the next statement is a `receive` statement. In Section 8 we thereafter show how our DPOR techniques can be extended to the situation where this assumption is not made, and processes can disable each other, e.g., by using locks or await statements.

3.2 Event Dependencies

A central concept in DPOR algorithms is that of a *happens-before relation* [Lamport 1978] between events in an execution sequence (also called a *causal relation* [Sen and Agha 2007]). We denote the happens-before relation in the execution sequence E by $\rightarrow_E$. Intuitively, for an execution sequence E, and two events e and e' in $dom(E)$, $e \rightarrow_E e'$ means that e "causally precedes" e' or that the ordering between e and e' may influence the outcome of the execution. For instance, e can be the transmission of a message that is received by e', or e can be a write operation to a shared variable that is accessed by e', or e and e' can both write to the same shared variable.

Our algorithms assume a function (called a *happens-before assignment*), which assigns a "happens-before" relation to any execution sequence. In order not to restrict to a specific computation model, we take a general approach, where the happens-before assignment is only required to satisfy a set of natural properties, which are collected in Definition 3.2. As long as it satisfies these properties, its precision can vary. For instance, the happens-before assignment can let any transmission to a certain message buffer be causally related with a reception from the same buffer. However, better reduction can be attained if the assignment does not make the transmission of a message dependent with the reception of a different message.

In practice, the happens-before assignment function is implemented by relating accesses to the same variables, transmissions and receptions of the same messages, etc., typically using *vector clocks* [Mattern 1989]. In Section 10 we describe such an assignment, suitable for Erlang programs.

Definition 3.2 (Properties of valid happens-before relations). A happens-before assignment, which assigns a unique happens-before relation $\rightarrow_E$ to any execution sequence E, is *valid* if it satisfies the following properties for all execution sequences E.

(1) $\rightarrow_E$ is an irreflexive partial order on $dom(E)$, which is included in $<_E$.
(2) The execution steps of each process are totally ordered, i.e., $\langle p, i\rangle \rightarrow_E \langle p, i+1\rangle$ whenever $\langle p, i+1\rangle \in dom(E)$,
(3) If $E' \leq E$, then $\rightarrow_E$ and $\rightarrow_{E'}$ are the same on $dom(E')$.
(4) Any linearization E' of $\rightarrow_E$ on $dom(E)$ is an execution sequence which has exactly the same "happens-before" relation $\rightarrow_{E'}$ as $\rightarrow_E$. This means that the relation $\rightarrow_E$ induces a set of equivalent execution sequences, all containing the same set of events, and with the same "happens-before" relation. We use:
    - $E \simeq E'$ to denote that $dom(E) = dom(E')$ and that E and E' are linearizations of the same "happens-before" relation, and
    - $[E]_{\simeq}$ to denote the equivalence class of E.